Sistem Informasi Seluler

Widget/UI dalam Flutter

Pengenalan Widget dalam Dart untuk Pengembangan Aplikasi



Poin Pembelajaran

- Apa itu Flutter
- Struktur Flutter
- Invisible Widget
- Visible Widget
- Selection Widget

Apa itu Flutter?

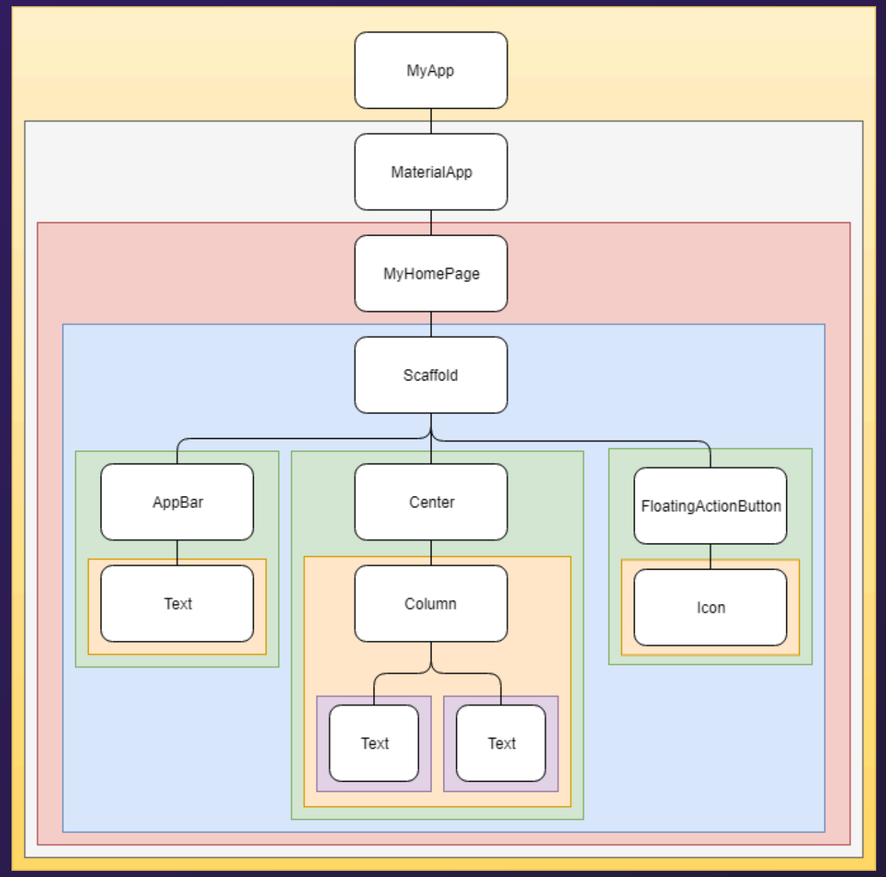
Apa itu Flutter?

Flutter adalah platform framework yang digunakan para developer untuk membuat aplikasi multiplatform hanya dengan satu codebase yaitu aplikasi yang dihasilkan dapat dipakai di berbagai platform, baik mobile Android, iOS, web, maupun desktop.

Untuk membangun tampilannya, Flutter menggunakan sistem yang disebut Widget, yang bisa dibayangkan seperti balok-balok LEGO, ada balok untuk tombol, tulisan, dan gambar yang disusun untuk membentuk keseluruhan aplikasi. Hasilnya, proses pengembangan menjadi jauh lebih cepat dan hemat, dengan keuntungan tambahan bahwa tampilan aplikasi akan terlihat sama persis dan berjalan dengan gesit di semua jenis perangkat

Sistem Informasi Seluler

Struktur Flutter



My app
- Berfungsi untuk menjalankan seluruh widget Flutter.

MaterialApp
- Memberikan tampilan dasar aplikasi bergaya Material Design, seperti tema, warna, dan navigasi.

MyHomePage
- Halaman utama aplikasi.

Scaffold
- Menyediakan area untuk AppBar, body, dan FloatingActionButton.

AppBar
- Bagian atas halaman berisi judul atau menu navigasi.
  - Text , Menampilkan teks seperti judul aplikasi.

Center
- Membuat widget di dalamnya berada di tengah layar.
  - Column, Menyusun widget secara vertikal (dari atas ke bawah).
    - Text (dalam Column),Menampilkan teks di bagian utama (body) aplikasi.

FloatingActionButton
- Biasanya digunakan untuk aksi utama, seperti tambah data.
  - Icon (dalam FloatingActionButton)
    - Menampilkan ikon di dalam tombol, misalnya tanda +.

Invisible Widget

Invisible Widget adalah widget-widget yang tidak memiliki wujud visual secara langsung, tetapi fungsinya sangat krusial sebagai kerangka dan pengatur tata letak. Sebagai contoh, Scaffold menyediakan struktur dasar sebuah halaman, sementara Row dan Column digunakan untuk menyusun widget-widget lain secara horizontal dan vertikal. Ada pula Container yang berfungsi sebagai kotak pembungkus serbaguna yang bisa diberi warna dan properti lain, serta Center yang tugasnya hanya menempatkan satu widget di tengah-tengah

Sistem Informasi Seluler

Material App

```dart
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Basic MaterialApp Example',
      theme: ThemeData(
        primarySwatch: Colors.blue, // Tema warna utama
      ),
      home: MyHomePage(), // Halaman utama aplikasi
    );
  }
}

```

MaterialApp adalah widget utama dalam Flutter yang digunakan untuk membangun aplikasi dengan gaya Material Design (gaya desain dari Google). Widget ini memberikan dasar untuk aplikasi, seperti mengatur tampilan tema, navigasi halaman, dan pengaturan bahasa.

Sistem Informasi Seluler

Scaffold

```dart
import 'package:flutter/material.dart';
void main() {
  runApp(MyApp());
}
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
// untuk pengisian appbar, body dll
),
    );
  }
}
```

Scaffold dalam Flutter adalah widget dasar untuk membangun struktur atau kerangka aplikasi dengan gaya Material Design. Scaffold menyediakan layout standar yang terdiri dari elemen-elemen seperti AppBar (bilah aplikasi), Drawer (menu navigasi), FloatingActionButton, BottomNavigationBar, dan Body (konten utama halaman).

Appbar

```dart
appBar: AppBar(
  title: Text('AppBar Example'),
),
```

AppBar adalah widget yang menampilkan bilah aplikasi (application bar) di bagian atas layar. Biasanya digunakan untuk menampilkan judul halaman, ikon, dan tombol aksi seperti ikon navigasi atau menu. AppBar juga bisa di-custom untuk memiliki background, teks, dan ikon yang sesuai dengan tema aplikasi.

Body

```
body: Center(
  child: Text('This is the Body'),
),
```

Body adalah bagian dari widget Scaffold yang menampung konten utama halaman. Anda bisa menempatkan berbagai widget di dalam body, seperti teks, gambar, tombol, atau layout lainnya. Bagian ini menjadi fokus utama di mana seluruh elemen UI ditempatkan.

Sistem Informasi Seluler

Drawer

```
drawer: Drawer(
  child: ListView(
    children: <Widget>[
      DrawerHeader(
        decoration: BoxDecoration(color: Colors.blue),
        child: Text('Header'),
      ),
      ListTile(
        leading: Icon(Icons.home),
        title: Text('Home'),
        onTap: () {
          // Aksi ketika item diklik
        },
      ),
    ],
  ),
),

```

Drawer adalah widget yang menampilkan menu navigasi samping yang biasanya diakses dengan menggeser layar dari kiri atau dengan menekan ikon menu di AppBar. Drawer sering digunakan untuk menampilkan daftar pilihan navigasi yang memungkinkan pengguna berpindah antar halaman atau fitur di dalam aplikasi.

BottomNavigation Bar

```dart
bottomNavigationBar: BottomNavigationBar(
  items: const <BottomNavigationBarItem>[
    BottomNavigationBarItem(icon: Icon(Icons.home), label: 'Home'),
    BottomNavigationBarItem(icon: Icon(Icons.business), label: 'Business'),
  ],
  currentIndex: 0,  // Index tab yang aktif
  onTap: (int index) {
    // Aksi saat item diklik
  },
),
```

BottomNavigationBar adalah widget yang menampilkan bilah navigasi di bagian bawah layar, biasanya dengan beberapa ikon atau teks yang dapat digunakan untuk berpindah antar halaman atau fitur. BottomNavigationBar umumnya digunakan dalam aplikasi dengan lebih dari satu halaman utama yang sering diakses, seperti aplikasi belanja atau media sosial.

Sistem Informasi Seluler

TabBar & TabBarView

```
appBar: AppBar(
  bottom: TabBar(
    tabs: [
      Tab(text: 'Tab 1'),
      Tab(text: 'Tab 2'),
    ],
  ),
),
body: TabBarView(
  children: [
    Center(child: Text('Content for Tab 1')),
    Center(child: Text('Content for Tab 2')),
  ],
),
```

TabBar dan TabBarView digunakan untuk membuat navigasi berbasis tab. TabBar adalah bagian yang menampilkan tab (misalnya, Tab 1, Tab 2), sementara TabBarView digunakan untuk menampilkan konten yang sesuai dengan setiap tab yang dipilih. Kombinasi ini sering digunakan ketika ada beberapa bagian konten atau fitur yang dapat dipilih pengguna tanpa berpindah halaman.

Sistem Informasi Seluler

Row

```
body: Row(
  mainAxisAlignment: MainAxisAlignment.center,
// Menyusun anak di tengah
  children: <Widget>[
    Text('Item 1'),
    SizedBox(width: 10), // Jarak antara item
    Text('Item 2'),
  ],
),
```

Row adalah widget layout yang digunakan untuk menempatkan anak-anak widget secara horizontal (dari kiri ke kanan). Setiap widget yang ada di dalam Row akan diatur dalam satu baris. Widget ini sangat berguna ketika Anda ingin menyusun elemen-elemen seperti ikon dan teks secara sejajar.

Column

```dart
body: Column(
  mainAxisAlignment: MainAxisAlignment.center, // Menyusun anak di tengah
  children: <Widget>[
    Text('Item 1'),
    SizedBox(height: 10), // Jarak antara item
    Text('Item 2'),
  ],
),
```

Column adalah kebalikan dari Row, di mana widget menempatkan anak-anaknya secara vertikal (dari atas ke bawah). Column biasanya digunakan ketika Anda ingin menampilkan elemen-elemen seperti teks, gambar, atau tombol secara bertingkat dari atas ke bawah.

Padding

```
1  body: Padding(
2    padding: EdgeInsets.all(16.0), // Padding di
   semua sisi
3    child: Text('Padded Text'),),
```

Padding adalah widget yang digunakan untuk menambahkan ruang di sekitar widget anak (child). Padding membantu dalam memberikan jarak antara widget dan batas luar widget lainnya. Anda bisa menentukan padding di semua sisi atau hanya pada sisi tertentu saja.

Container

```
body: Container(
  color: Colors.blue,     // Latar belakang biru
  width: 100,             // Lebar kotak
  height: 100,            // Tinggi kotak
  child: Center(
    child: Text('Inside Container'), // Konten di dalam container
  ),
),
dsadsads
```

Container adalah widget serbaguna yang dapat digunakan untuk membuat kotak pembungkus bagi widget lain. Container memungkinkan Anda untuk menambahkan properti seperti warna latar belakang, margin, padding, border, atau ukuran tertentu ke child-nya. Container sangat fleksibel dan sering digunakan untuk menyusun tata letak yang lebih kompleks.

Center

```
body: Center(
  child: Text('Centered Text'),
),
```

Center adalah widget yang memposisikan anak-anaknya (child) di tengah-tengah layar secara horizontal dan vertikal. Center sangat sederhana namun sangat berguna ketika Anda ingin meletakkan satu widget di tengah tanpa perlu mengatur tata letak secara manual.

Visible Widget

Visible Widget adalah elemen-elemen antarmuka yang bisa dilihat dan sering kali diinteraksikan langsung oleh user. Contohnya adalah Text, yang digunakan untuk menampilkan tulisan statis di layar, serta berbagai jenis tombol seperti ElevatedButton yang tampak terangkat untuk aksi utama, TextButton yang hanya berupa teks untuk aksi sekunder, dan IconButton yang menampilkan ikon sebagai tombol visual.

Text

```dart
body: Center(
  child: Text(
    'Hello, Flutter!',
    style: TextStyle(fontSize: 24, color: Colors.blue),
  ),
),
```

Widget Text adalah komponen fundamental yang berfungsi untuk menampilkan teks statis di layar aplikasi. Teks ini bisa berupa berbagai hal, mulai dari tulisan biasa, judul halaman, hingga label untuk komponen lain. Untuk menyesuaikan dengan desain aplikasi, developer dapat dengan mudah mengatur berbagai atribut dari teks tersebut, seperti mengubah warna, ukuran font, gaya tulisan, serta perataannya.

ElevatedButton

```dart
body: Center(
  child: ElevatedButton(
    onPressed: () {
      print('ElevatedButton Pressed');
    },
    child: Text('Elevated Button'),
  ),
),
```

Untuk tindakan utama dalam sebuah aplikasi, ElevatedButton sering menjadi pilihan karena desainnya yang menonjol. Widget ini merupakan tombol dengan efek elevasi yang membuatnya tampak sedikit terangkat dari permukaan layar. Karena memberikan umpan balik visual yang kuat saat ditekan, tombol ini sangat cocok digunakan sebagai tombol untuk aksi-aksi terpenting dalam aplikasi.

FloatingActionButton

```dart
floatingActionButton: FloatingActionButton(
  onPressed: () {
    print('Floating Action Button Pressed');
  },
  child: Icon(Icons.add), // Ikon "+" sebagai simbol penambahan
),
```

FloatingActionButton adalah sebuah tombol khusus yang didesain untuk mengambang di atas konten utama aplikasi, dan umumnya ditempatkan di sudut kanan bawah layar. Tombol ini dirancang untuk mewakili tindakan paling utama dalam sebuah halaman atau aplikasi, seperti fungsi untuk menambah item baru atau membuka sebuah fitur penting.

TextButton

```
body: Center(
  child: TextButton(
    onPressed: () {
      print('TextButton Pressed');
    },
    child: Text('Text Button', style: TextStyle(color:
Colors.blue)),
  ),
),
```

Berbeda dari ElevatedButton, TextButton adalah tombol yang hanya menampilkan teks tanpa disertai efek elevasi sama sekali. Karena desainnya yang lebih sederhana dan tidak terlalu menonjol, widget ini sangat ideal untuk digunakan pada aksi sekunder atau tindakan tambahan, seperti pada link atau pilihan-pilihan lain yang tidak menjadi fokus utama pengguna.

Sistem Informasi Seluler

IconButton

```
body: Center(
  child: IconButton(
    icon: Icon(Icons.favorite),
    color: Colors.red,
    onPressed: () {
      print('IconButton Pressed');
    },
  ),
),
```

IconButton menawarkan cara yang lebih visual untuk merepresentasikan sebuah aksi, karena widget ini merupakan tombol yang menampilkan ikon, bukan tulisan. Tombol ini sangat efektif digunakan untuk tindakan-tindakan yang mudah dipahami melalui simbol, contohnya seperti ikon untuk menambahkan item, menyimpan data, ataupun menghapus sesuatu.

Sistem Informasi Seluler

DropdownButton

```dart
String dropdownValue = 'Option 1';
body: Center(
  child: DropdownButton<String>(
    value: dropdownValue,
    icon: Icon(Icons.arrow_downward),
    items: <String>['Option 1', 'Option 2', 'Option 3'].map((String value) {
      return DropdownMenuItem<String>(
        value: value,
        child: Text(value),
      );
    }).toList(),
    onChanged: (String? newValue) {
      dropdownValue = newValue!;
      print('Selected: $dropdownValue');
    },
  ),
),

```

Ketika user perlu memilih satu dari beberapa opsi yang tersedia, DropdownButton menjadi widget yang sangat berguna. Saat tombol ini ditekan, sebuah daftar pilihan akan muncul di bawahnya, memungkinkan user untuk memilih salah satu opsi dari daftar tersebut. Setelah pilihan dibuat, daftar akan kembali tertutup dan menampilkan nilai yang telah dipilih.

Selection Widget

Selection Widget adalah kumpulan komponen interaktif yang dirancang khusus untuk memungkinkan user membuat pilihan dari berbagai opsi yang tersedia.

Sistem Informasi Seluler

Checkbox

```dart
bool isChecked = false;
body: Center(
  child: Checkbox(
    value: isChecked,
    onChanged: (bool? newValue) {
      isChecked = newValue!;
      print('Checkbox: $isChecked');
    },
  ),
),
```

Widget Checkbox memberikan user kemampuan untuk memilih atau membatalkan sebuah pilihan dengan cara mencentang atau menghapus centang pada sebuah kotak kecil. Fungsi ini sangat berguna dalam situasi di mana terdapat beberapa pilihan yang dapat dipilih secara bersamaan oleh user.

Sistem Informasi Seluler

RadioButton

```
int selectedValue = 1;
body: Column(
  children: [
    Radio<int>(
      value: 1,
      groupValue: selectedValue,
      onChanged: (int? newValue) {
        selectedValue = newValue!;
        print('Selected Radio: $selectedValue');
      },
    ),
    Radio<int>(
      value: 2,
      groupValue: selectedValue,
      onChanged: (int? newValue) {
        selectedValue = newValue!;
        print('Selected Radio: $selectedValue');
      },),],),

```

Berbeda dengan Checkbox, widget Radio digunakan ketika user harus memilih satu opsi dari beberapa pilihan yang tersedia dalam satu kelompok. Dalam implementasinya, hanya satu pilihan Radio yang bisa aktif dalam satu waktu, sehingga memastikan user hanya dapat membuat satu pilihan eksklusif dari seluruh opsi yang diberikan.

Sistem Informasi Seluler

Switch

```dart
bool isSwitched = false;
body: Center(
  child: Switch(
    value: isSwitched,
    onChanged: (bool newValue) {
      isSwitched = newValue;
      print('Switch: $isSwitched');
    },
  ),
),
```

Widget Switch berfungsi sebagai pengubah antara dua keadaan, yang umumnya adalah kondisi on dan off. Tampilannya yang menyerupai tombol saklar membuatnya sangat intuitif untuk digunakan dalam menandai pilihan biner, seperti untuk mengatur status aktif/tidak aktif atau menyalakan/mematikan sebuah fitur.

Sistem Informasi Seluler

Slider

```dart
double sliderValue = 0.5;

body: Center(
  child: Slider(
    value: sliderValue,
    min: 0,
    max: 1,
    divisions: 10,  // Jumlah partisi
    label: sliderValue.toString(),
    onChanged: (double newValue) {
      sliderValue = newValue;
      print('Slider Value: $sliderValue');
    },
  ),
),
```

Slider adalah widget yang memungkinkan user untuk memilih sebuah nilai dalam rentang tertentu dengan cara menggeser penunjuk (thumb) di sepanjang jalur (track) yang tersedia. Widget ini sangat sering digunakan untuk memilih nilai numerik, seperti saat mengatur tingkat volume, kecerahan layar, atau menentukan ukuran tertentu.

DatePicker

```dart
Future<void> _selectDate(BuildContext context) async {
  DateTime? selectedDate = await showDatePicker(
    context: context,
    initialDate: DateTime.now(), // Tanggal default saat ini
    firstDate: DateTime(2000), // Tanggal terawal yang dapat dipilih
    lastDate: DateTime(2100),  // Tanggal terakhir yang dapat dipilih
  );

  if (selectedDate != null) {
    print('Selected date: $selectedDate');
  }
}
```

DatePicker berfungsi untuk menampilkan sebuah dialog kalender yang interaktif, di mana user dapat dengan mudah memilih tanggal yang diinginkan. Setelah tanggal dipilih dan pengguna menekan "OK", nilai tanggal tersebut akan dikembalikan oleh fungsi. Nilai ini kemudian dapat digunakan untuk berbagai keperluan dalam aplikasi, misalnya untuk pengisian data pada sebuah formulir.

TimePicker

```dart
Future<void> _selectTime(BuildContext context) async {
  TimeOfDay? selectedTime = await showTimePicker(
    context: context,
    initialTime: TimeOfDay.now(), // Waktu default saat ini
  );

  if (selectedTime != null) {
    print('Selected time: ${selectedTime.format(context)}');
  }
}
```

Mirip dengan DatePicker, TimePicker adalah widget yang memungkinkan user untuk memilih waktu, yaitu jam dan menit, melalui sebuah tampilan dialog yang menyerupai jam. Dialog ini menampilkan kontrol jam yang intuitif, sehingga user dapat memilih waktu secara manual dengan mudah.



Thank You, now it’s challenge time!